

Reviewer Pack — Protocol-Induced Covers on High-Asdim Gadget Lifts Require E...

Entry 52a971f74369 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-29 13:06:25 UTC

Protocol-Induced Covers on High-Asdim Gadget Lifts Require Exponential-in- $Q(f)$ Multiplicity

Entry ID: 52a971f74369 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-29 13:06:25 UTC

1. Conjecture

Field A (mathematical branch): Coarse Geometric Lifting (CGL) — Geometric Complexity Theory

Field B (complexity object): Deterministic 2-Party Communication Complexity ($D^{\{cc\}}$)

Statement:

For any family of MetricGadgets G with $\text{asdim}_R(G) \geq d > 0$ at scale $R = \text{diam}(G)$, and for any function $f: \{0,1\}^n \rightarrow \{0,1\}$ with query complexity $Q(f)$, every deterministic protocol Π computing $f \circ G^n$ induces a ProtocolPullback that forms a RoeCover of multiplicity $m_\Pi \geq 2^{c \cdot Q(f)}$ for some constant $c > 0$ depending only on d and G .

Rationale (proposer's reasoning):

This conjecture tests A2 (Protocol $\rightarrow$ Cover) in conjunction with the target invariant's requirement that $CC(f \circ G^n) \geq Q(f) \cdot \log_2(\text{asdim}_R(G)+1)$. If high asymptotic dimension forces every protocol to induce a cover with multiplicity exponentially large in $Q(f)$, then the log-multiplicity lower bound directly recovers the desired communication complexity scaling. It leverages the geometric rigidity implied by positive asdim: protocols of low cost induce low-multiplicity covers, which cannot exist if the space is sufficiently 'spread out' at large scales.

Taxonomy category: LIFTING (status at proposal time:)

Framework membership: framework fw_6997a27304, role: elaboration

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 0b02c821cdd5a332

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import product

def matrix_multiplication(A, B):
    m = len(A)
    n = len(B[0])
    p = len(B)
    C = [[0 for _ in range(n)] for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for k in range(p):
                C[i][j] += A[i][k] * B[k][j]
    return C

def gaussian_elimination(A, b):
    n = len(b)
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]
        for j in range(i+1, n):
            factor = A[j][i] / A[i][i]
            A[j][i:] = [A[j][k] - factor * A[i][k] for k in range(i, n)]
            b[j] -= factor * b[i]
    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i+1, n))) / A[i][i]
    return x
```

```

def protocol_pullback(G, protocol):
    k = len(G)
    m = len(protocol)
    pullback = [[0 for _ in range(k)] for _ in range(k)]
    for i in range(k):
        for j in range(k):
            pullback[i][j] = protocol[(i // (k-1)) % m][(j // (k-1)) % m]
    return pullback

def compute_cover_multiplicity(pullback, R):
    k = len(pullback)
    multiplicity = 0
    for i in range(k):
        for j in range(k):
            if pullback[i][j] > 0:
                multiplicity += pullback[i][j]
    return multiplicity

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Parameters
    n = 3
    k_values = [2, 3, 4, 5]
    Q_f_values = list(range(1, n+1))

    metric_values = []
    instances_tested = 0

    for k in k_values:
        G = [[1 if i == j else 0 for j in range(k)] for i in range(k)]
        for Q_f in Q_f_values:
            f = [random.randint(0, 1) for _ in range(Q_f)]
            protocol = [[[random.randint(0, 1) for _ in range(k)] for _ in range(k)] for _ in range(Q_f)]

            pullback = protocol_pullback(G, protocol)
            R = sum(sum(row) for row in G)
            multiplicity = compute_cover_multiplicity(pullback, R)

            metric_values.append(multiplicity)
            instances_tested += 1

    mean_metric_value = sum(metric_values) / len(metric_values)
    std_metric_value = math.sqrt(sum((x - mean_metric_value) ** 2 for x in metric_values) / len(metric_values))

    conjecture_holds = all(x >= 2**((Q_f * n) - 1) for Q_f, x in zip(Q_f_values, metric_values))
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "Cover Multiplicity",
        "metric_value": mean_metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

```

```

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8cbb71d9.py", line 108, in <module>
    result = run_trial(seed)
              ~~~~~^~~~~~

  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8cbb71d9.py", line 83, in run_trial
    multiplicity = compute_cover_multiplicity(pullback, R)
                   ~~~~~^~~~~~

  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8cbb71d9.py", line 60, in compute_cover_multiplicity
    if pullback[i][j] > 0:
       ~~~~~^~~~~~

TypeError: '>' not supported between instances of 'list' and 'int'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed due to a TypeError, indicating an issue in the compute_cover_multiplicity function where a list was compared to an integer. No results were produced,

preventing any determination of support or falsification. | next: Debug the compute_cover_multiplicity function to resolve the TypeError by ensuring pullback[i][j] is an integer or comparable numeric type before attempting comparison with 0.

11. Audit log (LLM calls)

Total LLM calls: 6

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	preregistration	ollama_remote	qwen3:8b	0	0	35297
2	novelty	google	gemini-2.5-flash	0	0	7436
3	novelty	google	gemini-2.5-flash	0	0	5871
4	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19463
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13058
6	judge	google	gemini-2.5-flash	0	0	6799

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 87924 ms total latency. Provider mix: {'ollama_remote': 3, 'google': 3}

(full prompt+response transcripts available in *research/audit/52a971f74369.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/52a971f74369.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/52a971f74369.tar.gz* (if generated)